What will happen here is that the `<script>...</script>` text defined in the "template" data is written into the `p` child of the `body` and becomes a valid `script` element, which means the browser parses it as JavaScript. We're all done, and you can go ahead and request that. Use a web browser rather than `curl` to get the full effect:

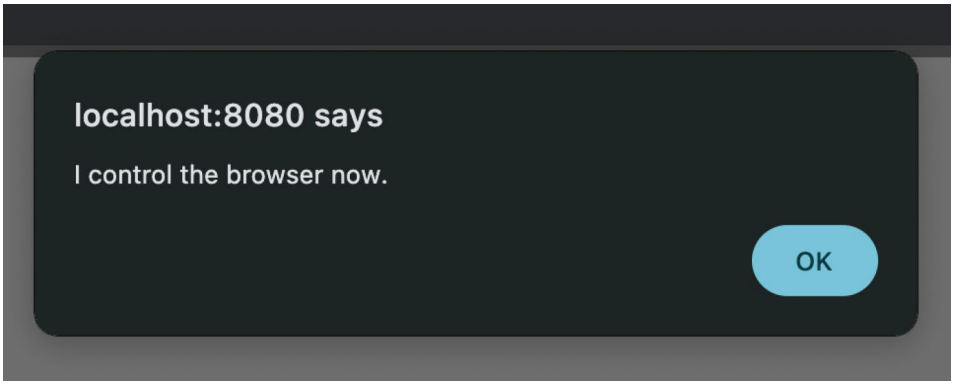


Figure 11-4. *Oops…*

Oh dear. What we have here is a classic Cross-Site Scripting (XSS) vulnerability. These happen when someone else is able to inject (usually malicious) code into your HTML, and there are a number of methods to achieve this, either in the client or via server. Naturally we're focusing on the server-side here, and one thing you really must do is NEVER TRUST DATA. I cannot stress this enough: if there is data flowing into your application from anywhere, either in the request itself or fetched from a database or API call, it must be treated with suspicion, *even if you (or your organization) own the data source.* Malicious employees exist; malicious users certainly exist. You must *filter* any data that is sent to the client.

Twig will (unless told otherwise) automatically escape HTML in any interpolated data, converting HTML meta-characters to named entities (and numeric entities where there are no named equivalents). It's worth noting that other PHP templating libraries do similar things too, which is another good reason for choosing one over vanilla PHP for outputting markup. Let's repeat the above example, but this time using a Twig template. Add the Twig autoloader/boilerplate and replace the entire `print`/heredoc statement with the following…

```
// ... $postData as before
require_once __DIR__ . '/vendor/autoload.php';

$loader = new \Twig\Loader\FilesystemLoader(__DIR__ . '/views');
```